The February 6, 2023 Kahramanmaraş Earthquake and Migration Dynamics

Damage, Displacement, and Recovery in 11 Earthquake-Affected Provinces

Şevval Miray Başar and Büşra Halil

2026-05-19

Abstract

Introduction

Data and Preprocessing

The final processed dataset is available here:
Download final RData file

Table 1: Summary of datasets used in the project.

Dataset	Content	Period	Main_Use
tr_nufus.xlsx	Annual province-level population by sex	2018–2025	Population trend and recovery analysis
deprem_hasar_tablosu.xlsx	Housing damage by province and damage severity	2023	Damage intensity calculation
deprem_2023_goc_akislari_duzenlenmis.xlsx	Cleaned 2023 migration flows from earthquake provinces	2023	Migration flow visualization
deprem_goc_ham_filtreli.xlsx	Annual inter-provincial migration data	2018–2025	Net migration rate analysis
İller_Arası_Göç.xlsx	Full 81x81 inter-provincial migration table	2008–2025	Supplementary Ankara analysis

Exploratory Data Analysis

Damage Intensity

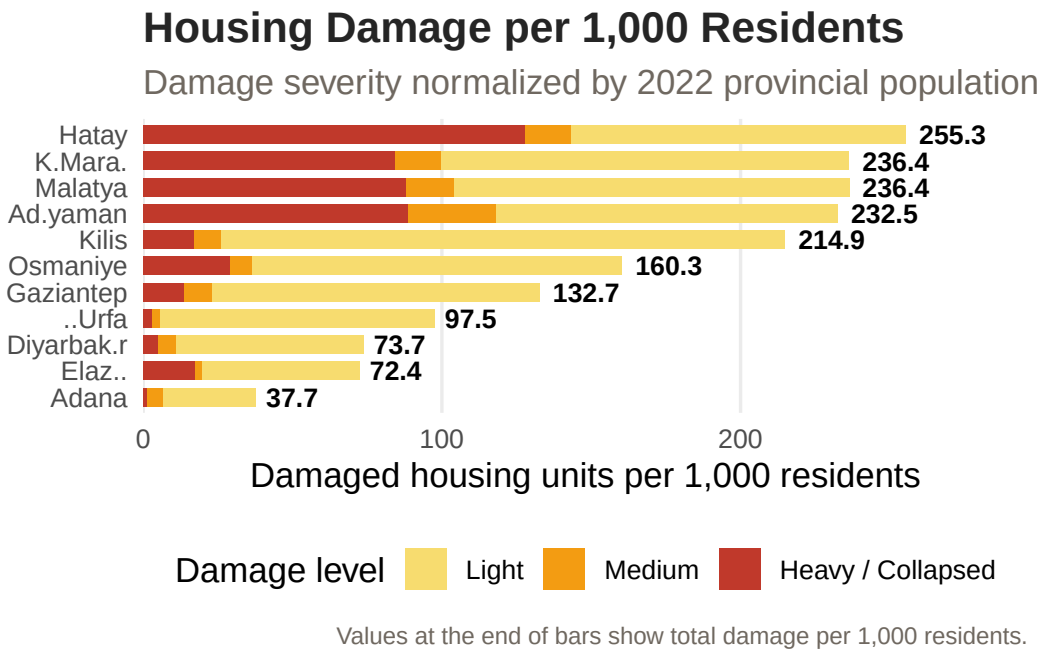

Figure 1: Housing damage intensity by province, normalized by 2022 population.

Migration Overview

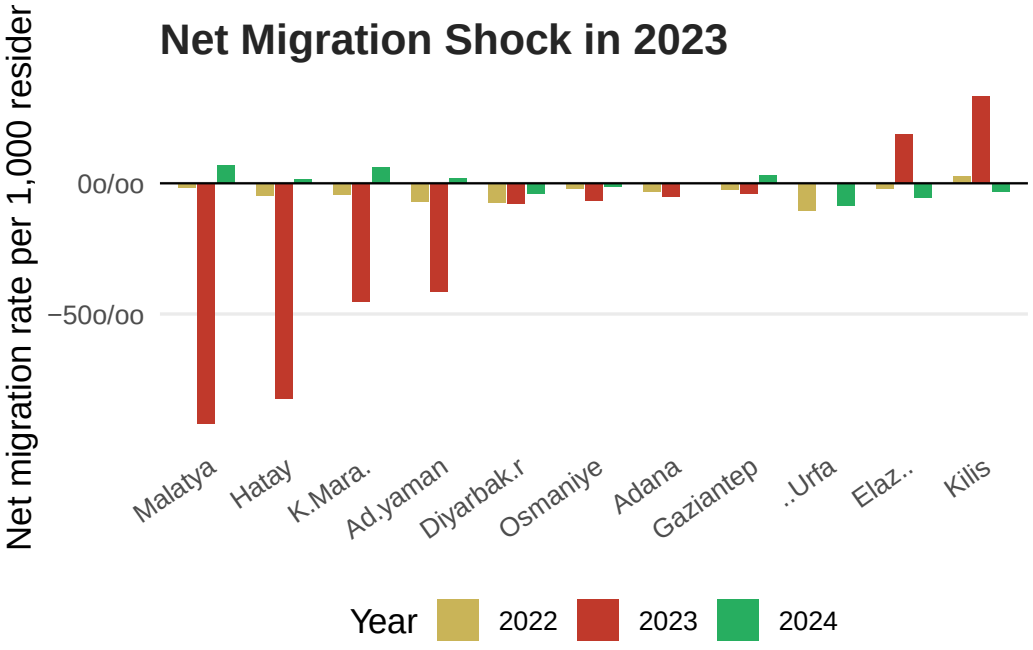

Figure 2: Net migration rate per 1,000 residents in 2022, 2023, and 2024.

Population Trends

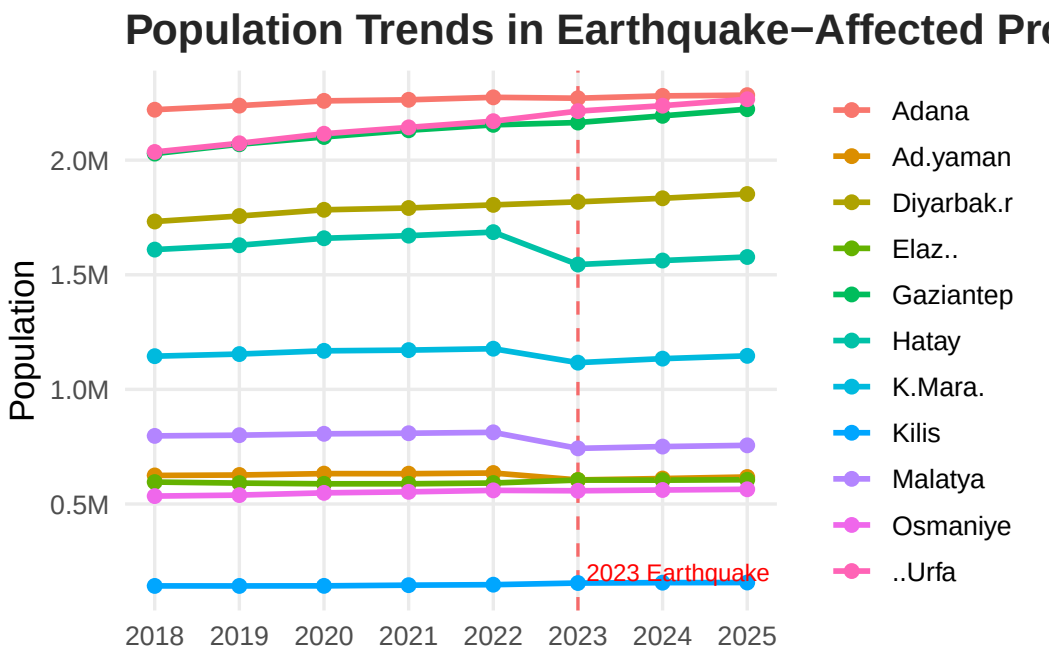

Figure 3: Population trend in 11 earthquake-affected provinces, 2018–2025.

Main Analysis

Interactive Migration Flow Map

```
coords <- tribble(
  ~province, ~lat, ~lon,
  "ADANA",37.00,35.32, "ADIYAMAN",37.76,38.28,
  "DIYARBAKIR",37.91,40.22, "ELAZIG",38.67,39.22,
  "GAZIANTEP",37.07,37.38, "HATAY",36.20,36.16,
  "KAHRAMANMARAS",37.58,36.92, "KILIS",36.72,37.12,
  "MALATYA",38.35,38.31, "OSMANIYE",37.07,36.25,
  "SANLIURFA",37.16,38.80, "ANKARA",39.93,32.86,
  "ISTANBUL",41.02,28.97, "MERSIN",36.80,34.64,
  "ANTALYA",36.90,30.70, "IZMIR",38.42,27.14,
  "KONYA",37.87,32.48, "KAYSERI",38.72,35.49,
  "MARDIN",37.31,40.74, "BURSA",40.18,29.06,
  "KOCAELI",40.77,29.95, "MUGLA",37.21,28.36,
  "AYDIN",37.85,27.84, "ESKISEHIR",39.77,30.52,
  "SAMSUN",41.29,36.33, "SIVAS",39.75,37.02
)

labels <- tribble(
  ~province, ~label,
  "ADANA","Adana", "ADIYAMAN","Adıyaman",
  "DIYARBAKIR","Diyarbakır", "ELAZIG","Elazığ",
  "GAZIANTEP","Gaziantep", "HATAY","Hatay",
  "KAHRAMANMARAS","K.Maraş", "KILIS","Kilis",
```

```
  "MALATYA","Malatya", "OSMANIYE","Osmaniye",
  "SANLIURFA","Ş.Urfa", "ANKARA","Ankara",
  "ISTANBUL","İstanbul", "MERSIN","Mersin",
  "ANTALYA","Antalya", "IZMIR","İzmir",
  "KONYA","Konya", "KAYSERI","Kayseri",
  "MARDIN","Mardin", "BURSA","Bursa",
  "KOCAELI","Kocaeli", "MUGLA","Muğla",
  "AYDIN","Aydın", "ESKISEHIR","Eskişehir",
  "SAMSUN","Samsun", "SIVAS","Sivas"
)

flows_map <- flows_2023 |>
  filter(value >= 500, to %in% coords$province) |>
  group_by(from_key = from, to_key = to) |>
  summarise(value = sum(value), .groups = "drop")

tr_simple <- tr_map |>
  st_transform(4326) |>
  st_simplify(dTolerance = 0.03) |>
  left_join(damage |> select(province, damage = heavy_per_1000), by = "province")

map_geojson <- geojsonsf::sf_geojson(tr_simple)
flow_json <- jsonlite::toJSON(flows_map, auto_unbox = TRUE)
coord_json <- jsonlite::toJSON(coords, auto_unbox = TRUE)
label_json <- jsonlite::toJSON(labels, auto_unbox = TRUE)

button_list <- c("ALL", EQ_KEYS)
button_labels <- c("All", labels$label[match(EQ_KEYS, labels$province)])

buttons_html <- paste0(
  paste0(
    "<button class='prov-btn", ifelse(button_list == "ALL", " active", ""),
    "' onclick=\"setFilter('", button_list, "', this)\">",
    button_labels, "</button>"
  ),
  collapse = ""
)

HTML(glue::glue('
<style>
#controls {{ display:flex; gap:8px; flex-wrap:wrap; align-items:center; margin-bottom:8px; fon
.prov-btn {{ font-size:11px; padding:4px 9px; border-radius:18px; cursor:pointer; border:1px s
.prov-btn.active {{ border-color:#8b1e1e; background:#fdecea; color:#8b1e1e; font-weight:bold;
#tooltip {{ position:fixed; background:white; border:1px solid #ccc; border-radius:8px; paddin
#legend {{ display:flex; gap:16px; font-size:12px; margin-top:8px; flex-wrap:wrap; }}
.leg-item {{ display:flex; align-items:center; gap:5px; }}
.leg-dot {{ width:11px; height:11px; border-radius:50%; }}
</style>
<div id="tooltip"></div>
<div id="controls"><b>Filter by origin:</b>{buttons_html}</div>
<svg id="mapsvg" width="100%" viewBox="0 0 680 420" style="display:block">
  <g id="map-layer"></g><g id="flows-layer"></g><g id="dots-layer"></g><g id="labels-layer"></
</svg>
```

```
<div id="legend">
  <div class="leg-item"><div class="leg-dot" style="background:#f0d27a"></div><span>Low damage
  <div class="leg-item"><div class="leg-dot" style="background:#e17035"></div><span>Medium dam
  <div class="leg-item"><div class="leg-dot" style="background:#4b000f"></div><span>High damag
  <div class="leg-item"><div class="leg-dot" style="background:#1a5276"></div><span>Destinatio
</div>
<script>
const TURKEY = {map_geojson};
const FLOWS = {flow_json};
const COORDS_RAW = {coord_json};
const LABELS_RAW = {label_json};
const COORDS = Object.fromEntries(COORDS_RAW.map(d => [d.province, [d.lat, d.lon]]));
const LABELS = Object.fromEntries(LABELS_RAW.map(d => [d.province, d.label]));
const LON_MIN=25.5, LON_MAX=45.2, LAT_MIN=35.4, LAT_MAX=42.6;
const SVG_X_MIN=35, SVG_X_MAX=610, SVG_Y_MIN=38, SVG_Y_MAX=315;

function toSVG(lat, lon) {{
  const x = SVG_X_MIN + (lon - LON_MIN) / (LON_MAX - LON_MIN) * (SVG_X_MAX - SVG_X_MIN);
  const y = SVG_Y_MAX - (lat - LAT_MIN) / (LAT_MAX - LAT_MIN) * (SVG_Y_MAX - SVG_Y_MIN);
  return [x, y];
}}

function damageColor(v) {{
  if (v == null || isNaN(v)) return "#d6d6d6";
  if (v < 10) return "#f0d27a";
  if (v < 30) return "#e69f45";
  if (v < 60) return "#d95f0e";
  if (v < 90) return "#b30000";
  return "#4b000f";
}}

function drawMap() {{
  const ml = document.getElementById("map-layer");
  TURKEY.features.forEach(f => {{
    const damage = f.properties.damage;
    const coords = f.geometry.coordinates;
    const polys = f.geometry.type === "MultiPolygon" ? coords : [coords];
    polys.forEach(poly => {{ poly.forEach(ring => {{
      let d = "";
      ring.forEach((pt, i) => {{
        const [x, y] = toSVG(pt[1], pt[0]);
        d += (i === 0 ? "M" : "L") + x.toFixed(1) + " " + y.toFixed(1) + " ";
      }});
      d += "Z";
      const path = document.createElementNS("http://www.w3.org/2000/svg","path");
      path.setAttribute("d", d);
      path.setAttribute("fill", damageColor(damage));
      path.setAttribute("fill-opacity", damage == null ? "0.48" : "0.88");
      path.setAttribute("stroke", "#ffffff");
      path.setAttribute("stroke-width", "0.7");
      ml.appendChild(path);
    }}); }});
  }});
```

```
}}

function dotR(v, maxDest) {{ return 5 + (v / maxDest) * 13; }}

function curvePath(x1, y1, x2, y2) {{
  const mx=(x1+x2)/2, my=(y1+y2)/2;
  const dx=x2-x1, dy=y2-y1, len=Math.sqrt(dx*dx+dy*dy);
  const bulge=Math.min(len*0.22,45);
  const cx=mx + (-dy/len)*bulge;
  const cy=my + ( dx/len)*bulge;
  return `M ${{x1.toFixed(1)}} ${{y1.toFixed(1)}} Q ${{cx.toFixed(1)}} ${{cy.toFixed(1)}} ${{x
}}

let activeFilter = "ALL";

function setFilter(p, btn) {{
  activeFilter = p;
  document.querySelectorAll(".prov-btn").forEach(b => b.classList.remove("active"));
  btn.classList.add("active");
  render();
}}

const tooltip = document.getElementById("tooltip");

function showTip(e, html) {{ tooltip.innerHTML = html; tooltip.style.opacity = 1; moveTip(e);
function moveTip(e) {{ tooltip.style.left = (e.clientX + 14) + "px"; tooltip.style.top = (e.cl
function hideTip() {{ tooltip.style.opacity = 0; }}

function render() {{
  const fl = document.getElementById("flows-layer");
  const dl = document.getElementById("dots-layer");
  const ll = document.getElementById("labels-layer");
  fl.innerHTML = ""; dl.innerHTML = ""; ll.innerHTML = "";

  const visible = activeFilter === "ALL" ? FLOWS : FLOWS.filter(f => f.from_key === activeFilt
  const destTotals = {{}};

  visible.forEach(f => destTotals[f.to_key] = (destTotals[f.to_key] || 0) + f.value);
  const maxDest = Math.max(...Object.values(destTotals), 1);

  visible.forEach(f => {{
    if (!COORDS[f.from_key] || !COORDS[f.to_key]) return;
    const [lat1, lon1] = COORDS[f.from_key];
    const [lat2, lon2] = COORDS[f.to_key];
    const [x1, y1] = toSVG(lat1, lon1);
    const [x2, y2] = toSVG(lat2, lon2);

    const path = document.createElementNS("http://www.w3.org/2000/svg","path");
    path.setAttribute("d", curvePath(x1,y1,x2,y2));
    path.setAttribute("fill", "none");
    path.setAttribute("stroke", "#8b1e1e");
    path.setAttribute("stroke-width", "1.8");
    path.setAttribute("stroke-opacity", "0.52");
```

```
    path.setAttribute("stroke-linecap", "round");
    path.addEventListener("mouseenter", e => showTip(e, `<b>${{LABELS[f.from_key]}} → ${{LABEL
    path.addEventListener("mousemove", moveTip);
    path.addEventListener("mouseleave", hideTip);
    fl.appendChild(path);
  }});

  const allProvs = new Set();
  visible.forEach(f => {{ allProvs.add(f.from_key); allProvs.add(f.to_key); }});

  allProvs.forEach(p => {{
    if (!COORDS[p]) return;
    const [lat, lon] = COORDS[p];
    const [x, y] = toSVG(lat, lon);
    const isDest = destTotals[p] > 0;

    if (isDest) {{
      const dot = document.createElementNS("http://www.w3.org/2000/svg","circle");
      dot.setAttribute("cx", x); dot.setAttribute("cy", y);
      dot.setAttribute("r", dotR(destTotals[p], maxDest));
      dot.setAttribute("fill", "#1a5276");
      dot.setAttribute("opacity", "0.80");
      dot.setAttribute("stroke", "white");
      dot.setAttribute("stroke-width", "1.5");
      dot.addEventListener("mouseenter", e => showTip(e, `<b>${{LABELS[p]}}</b><br>${{Number(d
      dot.addEventListener("mousemove", moveTip);
      dot.addEventListener("mouseleave", hideTip);
      dl.appendChild(dot);
    }}

    const text = document.createElementNS("http://www.w3.org/2000/svg","text");
    text.setAttribute("x", x + 7);
    text.setAttribute("y", y - 7);
    text.setAttribute("font-size", "10");
    text.setAttribute("font-weight", "700");
    text.setAttribute("fill", "#111");
    text.setAttribute("stroke", "white");
    text.setAttribute("stroke-width", "2.8");
    text.setAttribute("paint-order", "stroke");
    text.textContent = LABELS[p] || p;
    ll.appendChild(text);
  }});
}}

drawMap();
render();
</script>
'))
```

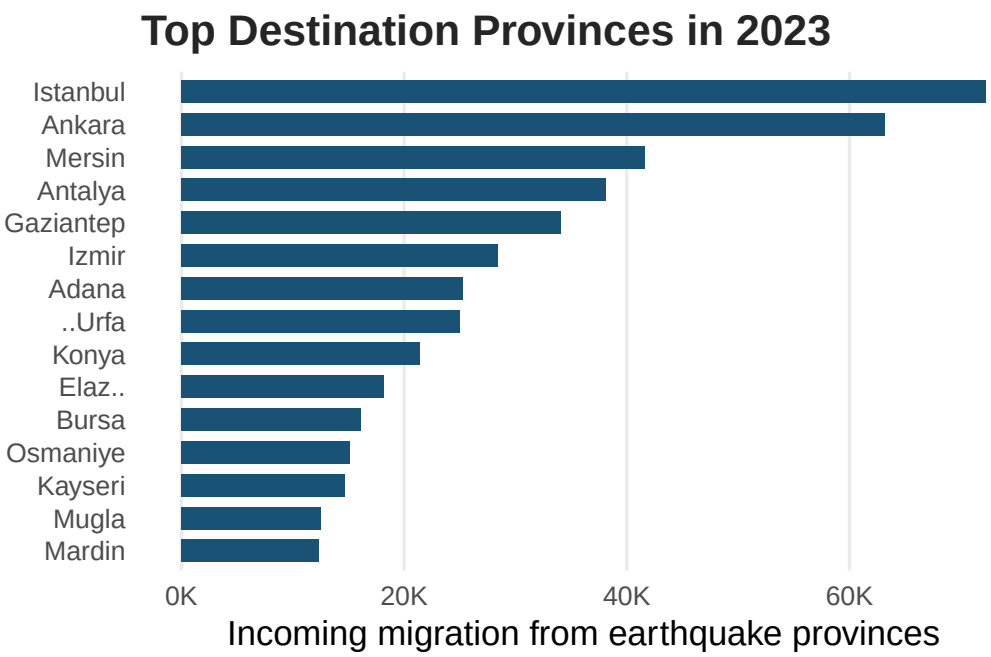

Figure 4: Top destination provinces receiving migrants from earthquake provinces in 2023.

Top Receiving Provinces

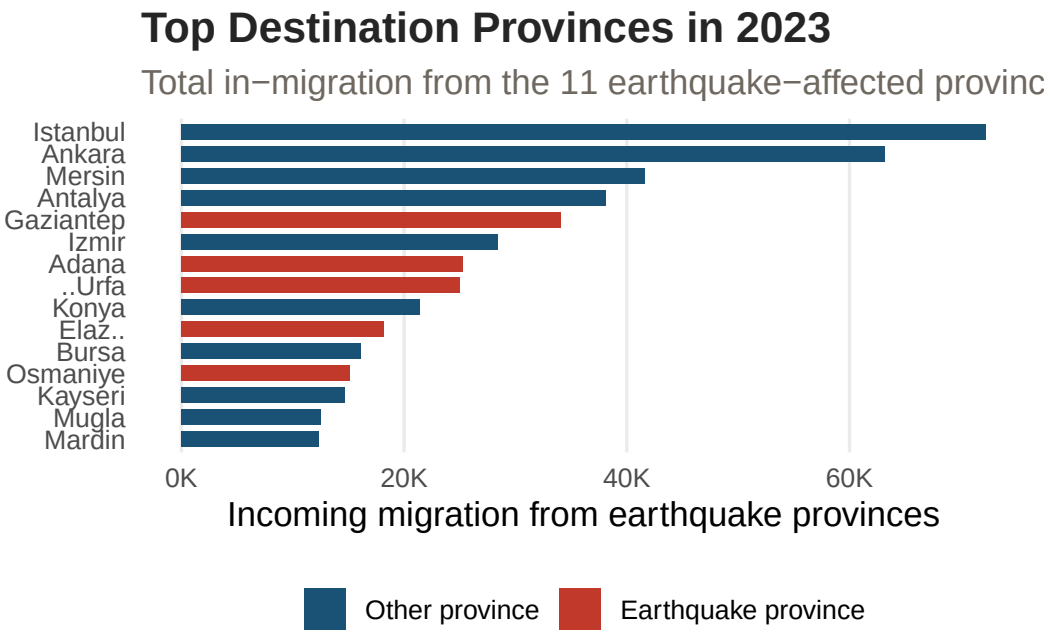

Figure 5: Top destination provinces receiving migrants from earthquake provinces in 2023.

Migration Routes from the Three Hardest-Hit Provinces

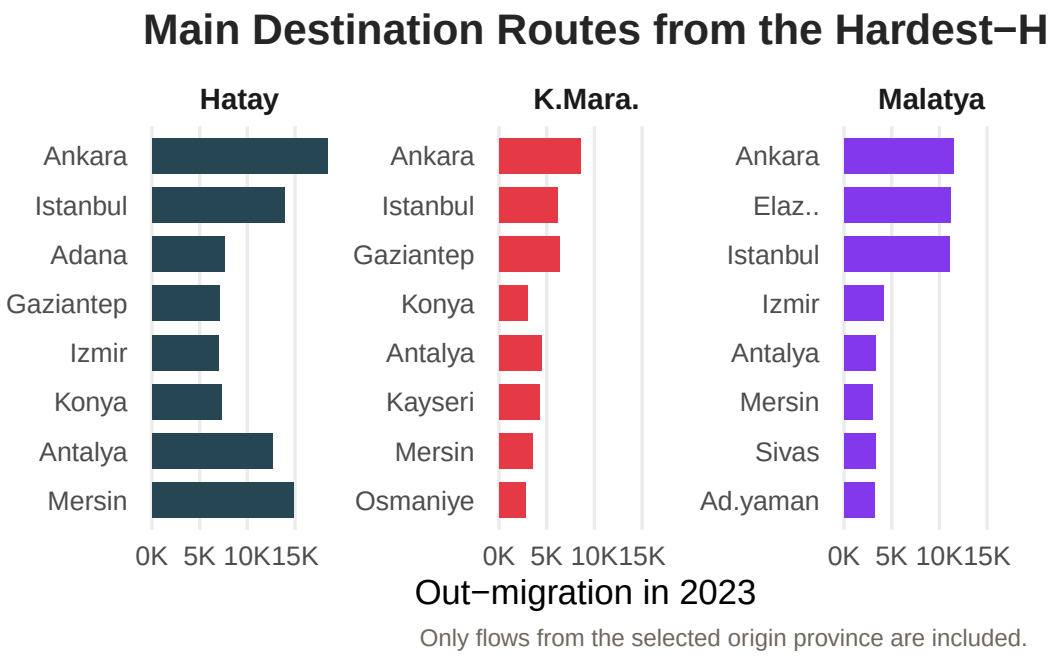

Figure 6: Top destination provinces for out-migrants from Hatay, Malatya and Kahramanmaraş in 2023.

Damage Intensity and Population Loss

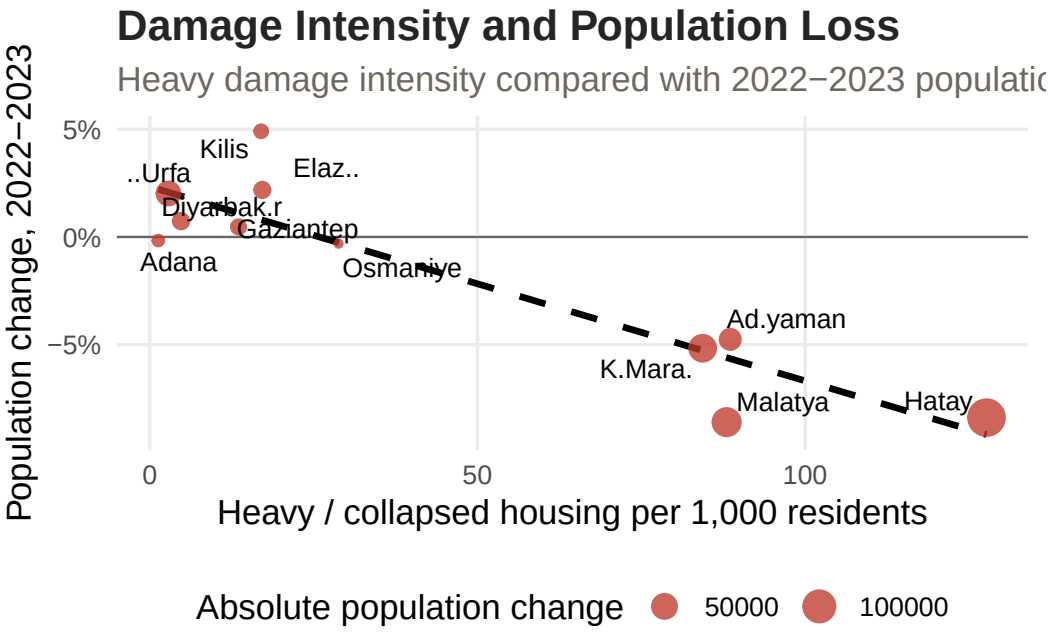

Figure 7: Relationship between heavy damage intensity and population change from 2022 to 2023.

Recovery Status by 2025

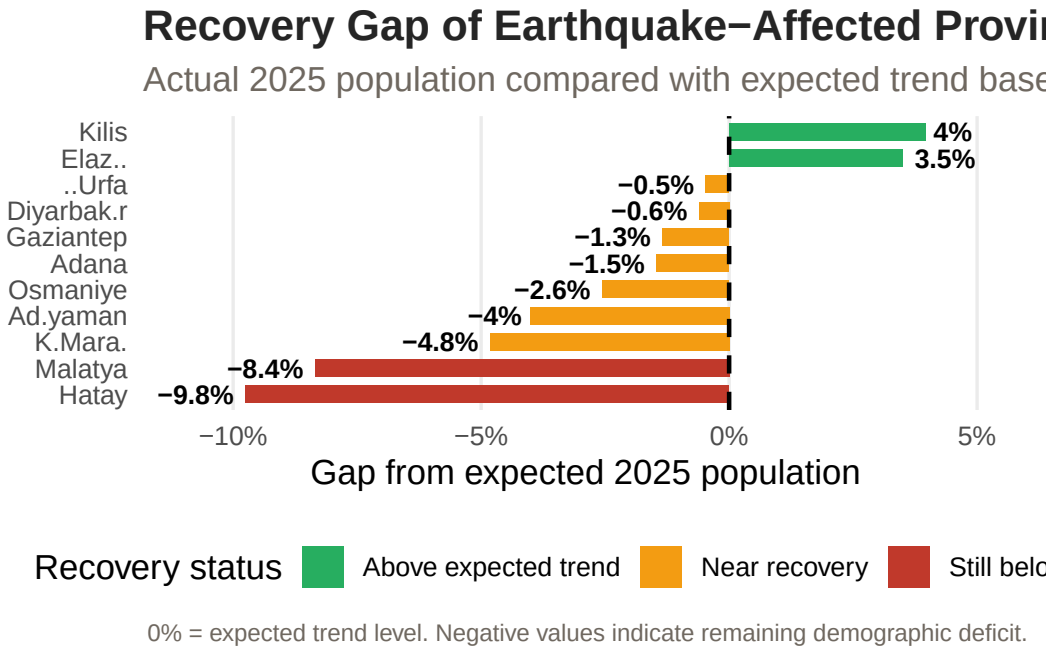

Figure 8: Recovery gap in 2025 relative to expected population based on 2018–2022 trend.

Supplementary Analysis

Ankara Migration Question

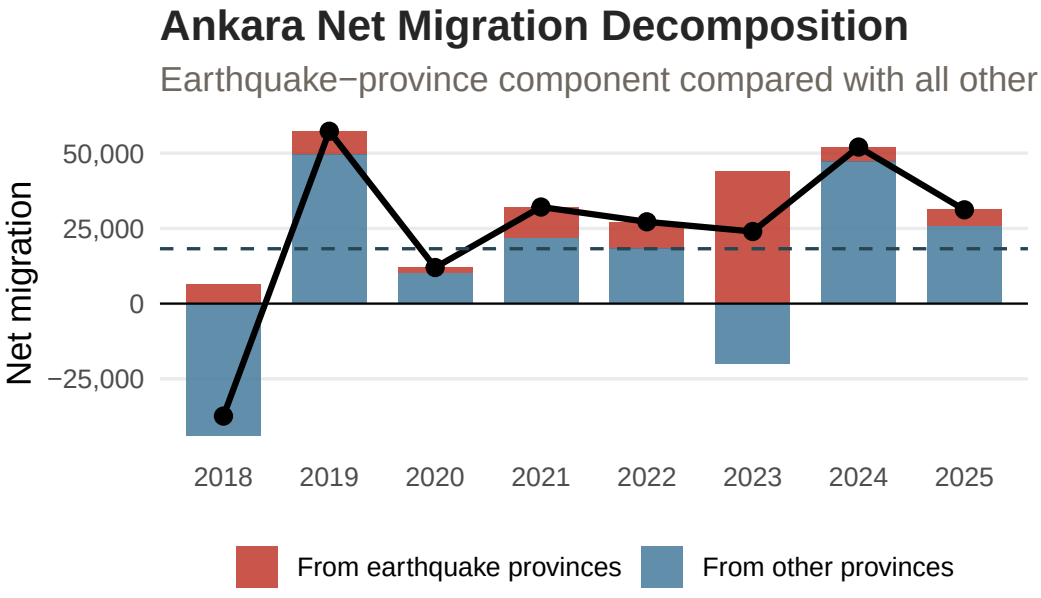

Figure 9: Ankara's net migration from earthquake provinces and other provinces, 2018–2025.

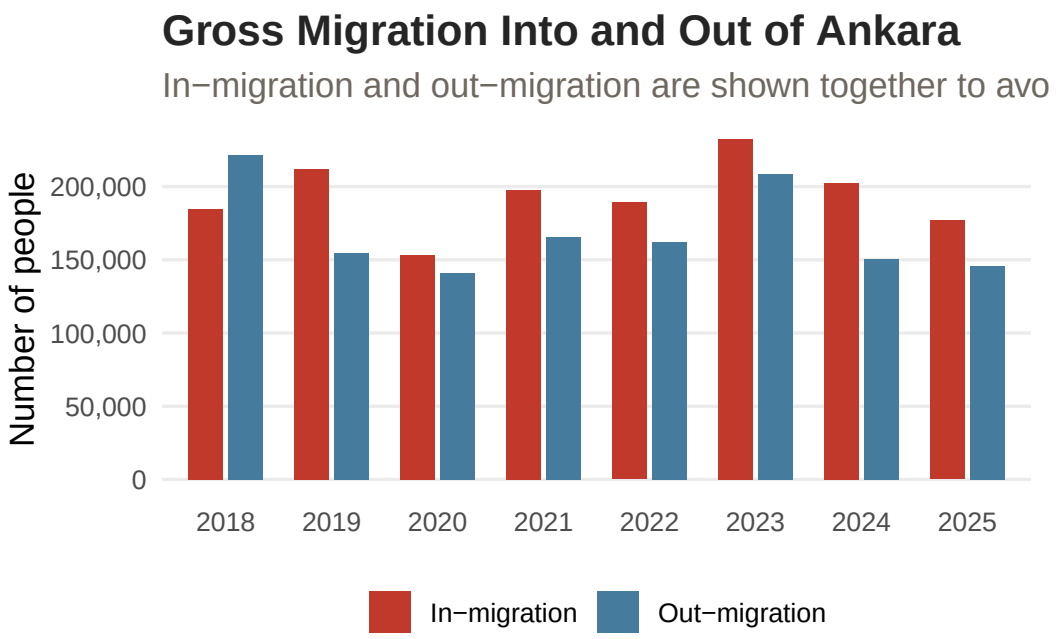

Figure 10: Gross in-migration and out-migration into/from Ankara, 2018–2025.

Table 2: Migration from each earthquake province into Ankara in 2023.

Province	In-Migration	Out-Migration	Net Migration	Share of EQ inflow (%)
Hatay	18356	2153	16203	29.0
Malatya	11470	1649	9821	18.2
K.Maraş	8564	1519	7045	13.6
Adıyaman	4136	974	3162	6.5
Adana	5508	2763	2745	8.7
Gaziantep	4197	2536	1661	6.6
Diyarbakır	3588	2476	1112	5.7
Osmaniye	2026	982	1044	3.2
Ş.Urfa	3299	2604	695	5.2
Elazığ	1764	1235	529	2.8
Kilis	284	339	-55	0.4

Key Takeaways

1. Damage intensity was not evenly distributed across the earthquake region; Hatay, Malatya, Adıyaman, and Kahramanmaraş stood out with the highest heavy-damage burden per 1,000 residents.
2. The strongest population losses in 2023 were observed in provinces with higher heavy-damage intensity, suggesting a visible relationship between physical destruction and demographic decline.
3. Migration flows from earthquake-affected provinces were directed not only toward major metropolitan areas such as Istanbul, Ankara, and Antalya, but also toward nearby regional destinations.
4. Recovery by 2025 remained uneven; some provinces approached or exceeded their expected population trend, while the hardest-hit provinces still showed a demographic gap.
5. The Ankara-specific analysis shows that earthquake-origin inflows increased in 2023, but total net migration should be interpreted together with outflows and other provincial movements.

Conclusion

References

Turkish Statistical Institute (TÜİK). (2026). *Address Based Population Registration System Results, 2025*. TÜİK Data Portal. Retrieved from the TÜİK official data portal.

Turkish Statistical Institute (TÜİK). (2026). *Internal Migration Statistics*. TÜİK Data Portal. Retrieved from the TÜİK official data portal.

Disaster and Emergency Management Presidency (AFAD). (2023). *February 6, 2023 Kahramanmaraş Earthquakes and Damage Assessment Data*. Republic of Türkiye Ministry of Interior, AFAD.

Presidency of Strategy and Budget. (2023). *2023 Kahramanmaraş and Hatay Earthquakes Report*. Republic of Türkiye Presidency of Strategy and Budget.

Alpers, K. (n.d.). *Turkey Maps GeoJSON: tr-cities.json*. GitHub repository.